國立臺北科技大學
National Taipei University of Technology

電子工程系碩士班

碩士學位論文

Department of Electronic Engineering

Master Thesis

非合作式 PCI 設備模擬

Non-cooperative PCI Device Emulation

研究生：蘇翊端

Researcher: Yi-Tuan Su

指導教授：賴冠廷 博士

Advisor: Kuan-Ting Lai, Ph.D.

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

審定書不用頁碼

摘要

關鍵詞：虛擬化、設備模擬、VirtIO、PCI transport

輕量虛擬機器透過最小化映像檔來降低資源佔用與攻擊面，但同時移除了除錯與監控等工具。vmsh 針對此問題提出非合作式（non-cooperative）的解決方案，能在不修改虛擬機器監視器（hypervisor）與客體作業系統（Guest OS）的前提下，將 VirtIO 設備動態掛載至運行中的虛擬機器，為其補充所需的服務與工具。然而，vmsh 與 Cloud Hypervisor 在中斷機制上存在不相容，限制了 vmsh 的 hypervisor 支援範圍。

本研究擴充 vmsh 的功能，在維持非合作式與 hypervisor 無關（hypervisor-agnostic）的特性前提下，為 vmsh 實現了 VirtIO over PCI transport，並利用 KVM 所提供的 KVM_SIGNAL_MSI 介面實現 MSI-X 中斷的注入，來解決 vmsh 在中斷機制上與 Cloud Hypervisor 的不相容問題。在 PCI 設備模擬方面，本研究延伸 vmsh 的 side-load library 機制在客體作業系統內部觸發 PCI rescan，並透過 ptrace 攔截客體作業系統發出的 Port I/O 指令，使其得以發現由 vmsh 所模擬的 PCI 虛擬設備，進而透過標準 virtio-pci 驅動程式與該設備進行 I/O 操作。

此外，本研究在 QEMU 與 Cloud Hypervisor 兩種環境下完成功能驗證，並使用 xfstests 確認本研究中 PCI transport 實現的初步正確性，使 vmsh 成功支援 Cloud Hypervisor，擴展了 vmsh 的 hypervisor 支援範圍，並驗證了使用非合作式方法模擬 PCI 設備的可行性。

ABSTRACT

Keywords: Virtualization, Device Emulation, VirtIO, PCI Transport

Lightweight virtual machines reduce resource consumption and attack surface by minimizing their images, but at the cost of removing debugging and monitoring tools. vmsh addresses this problem with a non-cooperative approach that dynamically attaches VirtIO devices to running virtual machines without modifying the hypervisor or the guest OS, supplementing VMs with the required services and tools. However, vmsh is incompatible with Cloud Hypervisor due to differences in interrupt mechanisms, limiting the range of hypervisors that vmsh can support.

This study extends vmsh's functionality by implementing VirtIO over PCI transport while preserving its non-cooperative and hypervisor-agnostic properties. This study leverages the KVM_SIGNAL_MSI system call interface provided by KVM to inject MSI-X interrupts, resolving the interrupt mechanism incompatibility between vmsh and Cloud Hypervisor. For PCI device emulation, this study extends vmsh's side-load library mechanism to trigger PCI rescan within the guest OS, and intercepts Port I/O instructions issued by the guest via ptrace, enabling the guest OS to discover the PCI virtual device emulated by vmsh and perform I/O operations through the standard virtio-pci driver.

Furthermore, this study conducts functional verification under both QEMU and Cloud Hypervisor environments, and uses xfstests to confirm the preliminary correctness of the PCI transport implementation. As a result, vmsh successfully supports Cloud Hypervisor, expanding its hypervisor compatibility and demonstrating the feasibility of emulating PCI devices using a non-cooperative approach.

TAIPEI TECH

Acknowledgements

所有對於研究提供協助之人或機構，作者都可在誌謝中表達感謝之意。

TAIPEI TECH

Table of Contents

摘要 i

ABSTRACT ii

Acknowledgements iii

Table of Contents iv

List of Figures vi

List of Tables vii

Chapter 1 Introduction 1

1.1 Background 1

1.2 Motivations 2

1.3 Research Contribution 3

1.4 Thesis Organization 4

Chapter 2 Literature Review 5

2.1 Virtualization and KVM 5

2.1.1 Hardware-Assisted Virtualization 5

2.1.2 Kernel-based Virtual Machine (KVM) 5

2.1.3 VM-Exit Mechanism 6

2.2 VirtIO Standard and PCI Transport 7

2.2.1 VirtIO Overview 7

2.2.2 Transport Mechanism 8

2.2.3 PCI Transport 8

2.3 VMSH: Hypervisor-agnostic Guest Overlays for VMs 10

2.3.1 Overview and Design Goals 10

2.3.2 System Architecture 11

2.3.3 Design and Key Techniques 13

2.4 Interrupt Mechanisms in Virtualization 15

2.4.1 KVM Interrupt Delivery Mechanism 15

2.4.2 MSI/MSI-X Interrupts 16

TAIPEI TECH

Chapter 3 Proposed Method 18
3.1 System Architecture Overview 18
3.2 PCI Bus Rescan and Device Discovery 22
3.2.1 PCI Rescan Trigger Mechanism 23
3.2.2 Rescan Process and Virtual Device Discovery 24
3.3 Port I/O Interception Mechanism 25
3.3.1 Two Types of I/O Exits 25
3.3.2 Port I/O Interception Handling 26
3.3.3 Response Protection Mechanism 27
3.4 VirtIO-PCI Device Emulation 28
3.4.1 Initialization Process Overview 29
3.4.2 PCI Configuration Space Stage 29
3.4.3 VirtIO Configuration Stage 32
3.5 MSI-X Interrupt Injection 33
3.5.1 KVM_SIGNAL_MSI Interrupt Mechanism 33
3.5.2 MSI-X Table and Interrupt Vector Management 34
3.5.3 Interrupt Injection Implementation 35
Chapter 4 Experiment and Result 37
4.1 Functional Validation and Generality 37
4.1.1 PCI Device Discovery and Initialization 37
4.1.2 Block I/O and Filesystem Validation 37
4.1.3 Cloud Hypervisor Validation 37
References 40

TAIPEI TECH

List of Figures

2.1 vmsh System Architecture (Adapted from [1]) 12

3.1 vmsh Original Architecture (VirtIO-MMIO Transport) 18

3.2 vmsh's Existing irqfd Interrupt Path Unavailable on Cloud Hypervisor (Split IRQChip) 19

3.3 System Architecture (PCI Transport). Stars (⋆) Indicate Components Added by This Study. 20

3.4 Simplified PCI Rescan Logic in the Linux Kernel (drivers/pci/probe.c) 23

3.5 PCI Rescan Behavior with a vmsh Virtual PCI Device at BDF 00:0a.0 24

3.6 Illustration of Response Protection Mechanism 28

3.7 Two Stages of VirtIO-PCI Device Initialization 30

3.8 Virtual PCI Device Configuration Space Layout 31

3.9 A Complete I/O Round-Trip 36

4.1 dmesg log of PCI device discovery and BAR assignment (QEMU) 37

4.2 dmesg log of VirtIO negotiation completion (QEMU) 37

4.3 Guest userspace query of the vdb device 37

4.4 Block I/O read/write and data integrity test (QEMU) 38

4.5 ext4 filesystem mount test (QEMU) 38

4.6 Data integrity test (Cloud Hypervisor) 38

4.7 ext4 filesystem mount test (Cloud Hypervisor) 39

TAIPEI TECH

List of Tables

TAIPEI TECH

Chapter 1 Introduction

1.1 Background

Virtualization is a cornerstone of modern cloud computing. Cloud service providers consolidate multiple tenants onto a single physical host through virtual machines (VMs) to achieve resource sharing and isolation. Among the various virtualization solutions, KVM (Kernel-based Virtual Machine) in the Linux kernel leverages hardware-assisted virtualization to provide efficient full virtualization capabilities and has become the dominant virtualization API in the industry, widely adopted by major cloud providers.

As the demand for performance-sensitive workloads grows, the industry has seen a trend toward designing lightweight VMs. These approaches reduce the VM image size to lower memory footprint and accelerate startup time, making them suitable for rapid-deployment scenarios such as serverless computing, while also improving system reliability and security by reducing the attack surface. The key to achieving this lightweight design lies in minimizing the VM's root filesystem, which entails removing services that are not needed during normal operation, such as monitoring and debugging tools.

However, this lightweight strategy introduces a fundamental tension: when users need to debug, monitor, or attach additional services to a VM, these tools no longer exist in the VM image. Rebuilding the image is not only cumbersome but also requires restarting the VM, causing the loss of runtime state and diagnostic information. This trade-off between the advantages of lightweight VMs and the availability of functionality raises a central question: is it possible to dynamically extend the capabilities of a running VM without modifying its image?

To address this problem, Thalheim et al. proposed vmsh [1] at EuroSys 2022, a hypervisor-agnostic mechanism for extending VM functionality. The core idea of vmsh is to dynamically attach VirtIO devices to a running VM from the outside, without modifying the hypervisor's source code or altering the guest image. Specifically, vmsh traces the hypervisor process using Linux's ptrace mechanism and leverages the KVM interface to side-load code into the guest kernel, registering VirtIO block and console devices within the guest to provide capabilities

such as filesystem mounting and interactive terminal access. At the device emulation level, vmsh employs VirtIO MMIO transport as the communication channel between host and guest, and uses KVM's irqfd mechanism for interrupt injection. This approach has been successfully validated on multiple mainstream KVM hypervisors, including QEMU, Firecracker, crosvm, and kvmtool, demonstrating its cross-hypervisor generality.

1.2 Motivations

Despite vmsh's demonstrated cross-hypervisor generality, a gap remains in its actual support coverage. Among the five KVM hypervisors it targets, Cloud Hypervisor, initiated by Intel, is the sole exception that remains incompatible. The authors of vmsh explicitly noted in their paper:

> *VMSH is able to support 4 out of the 5 hypervisors. Cloud Hypervisor is the exception as it uses PCIe's MSI-X messages for its interrupt handling. Therefore, it is incompatible with MMIO as a VirtIO transport channel.* [1]

vmsh's existing dynamic device attachment is implemented using VirtIO MMIO transport, whose interrupt injection relies on the interrupt routing mechanism internal to the KVM kernel. However, Cloud Hypervisor adopts a different interrupt architecture, rendering the interrupt path that vmsh depends on inoperable and consequently making the entire MMIO transport approach unusable on the Cloud Hypervisor platform.

To address this limitation, the authors of vmsh also outlined a research direction: "We plan to extend VMSH to support VirtIO over PCI for Cloud Hypervisor." [1] This implies that the virtual devices dynamically attached by vmsh would no longer be presented via MMIO transport but would instead appear as PCI devices within the guest OS, using MSI-X interrupt injection to replace the incompatible interrupt path and thereby resolving the compatibility issue with Cloud Hypervisor.

However, implementing VirtIO over PCI in a non-cooperative environment is fundamentally different from PCI device emulation performed internally by a hypervisor. In a typical

virtualization architecture, PCI device emulation is handled within the hypervisor, which has full access to PCI bus topology information and can directly manage device registration, configuration space responses, and interrupt routing. By contrast, vmsh operates outside the hypervisor and cannot access these internal data structures and interfaces; it must achieve the same objectives through indirect means. Specifically, as an external process independent of the hypervisor, vmsh faces the following technical challenges: first, intercepting the guest's accesses to PCI configuration space to enable virtual device discovery and configuration; second, injecting MSI-X interrupts to replace the incompatible interrupt path; and third, implementing complete VirtIO-PCI device emulation, including the Capabilities structure, BAR mapping, and virtqueue management.

In summary, the design of vmsh pursues two core objectives: non-cooperativeness, requiring no pre-deployed programs in the guest userspace, and generality, not relying on the internal interfaces of any specific hypervisor. As discussed above, vmsh's existing MMIO transport approach cannot support Cloud Hypervisor, while implementing PCI device emulation from outside the hypervisor involves several unresolved technical challenges. Therefore, how to implement VirtIO over PCI transport while maintaining vmsh's non-cooperative and hypervisor-agnostic design philosophy, thereby enabling vmsh to support Cloud Hypervisor, is the central research question that this study aims to address.

1.3 Research Contribution

This study extends vmsh to address its incompatibility with Cloud Hypervisor by implementing VirtIO over PCI transport in a non-cooperative environment. The main contributions are as follows:

Non-cooperative PCI device emulation. As an external process independent of the hypervisor, vmsh must complete device emulation from the outside without hypervisor cooperation. This study extends vmsh's side-load library mechanism to trigger a PCI rescan within the guest OS and intercepts Port I/O instructions issued by the guest via ptrace, implementing PCI device discovery, configuration space emulation, BAR mapping, and VirtIO-PCI negotiation, enabling

the guest to access the device emulated by vmsh through the standard virtio-pci driver.

Resolving interrupt mechanism compatibility and extending hypervisor support. vmsh's existing interrupt injection mechanism relies on the Full IRQChip architecture and cannot operate under Cloud Hypervisor's Split IRQChip environment. This study adopts the KVM_SIGNAL_MSI interface provided by KVM to implement MSI-X interrupt injection, making the interrupt mechanism independent of IRQChip architecture differences.

Cross-hypervisor validation of correctness and generality. Functional validation is performed under both QEMU and Cloud Hypervisor environments to confirm the generality of PCI transport; additionally, correctness testing using xfstests is conducted under the QEMU environment to confirm the preliminary correctness of PCI transport.

1.4 Thesis Organization

Chapter 1 Introduction presents the background and motivation of this study, describes the incompatibility between vmsh and Cloud Hypervisor, and states the contributions of this research.

Chapter 2 Literature Review reviews the technical background relevant to this study, including the KVM virtualization architecture, the VirtIO device model, the PCI bus and MSI-X interrupt mechanism, and introduces the system architecture and design of vmsh.

Chapter 3 Proposed Method details the system design and implementation of this study, covering PCI device discovery, Port I/O interception, VirtIO-PCI device emulation, and MSI-X interrupt injection.

Chapter ?? Experiment and Results presents the validation results of this study, including functional validation under QEMU and Cloud Hypervisor, correctness testing with xfstests, and a discussion of the limitations of the prototype implementation.

Chapter ?? Conclusion and Future Work summarizes the results and contributions of this study, and proposes future research directions including an asynchronous I/O model, performance testing, and integration with vmsh's broader functionality.

TAIPEI TECH

Chapter 2 Literature Review

2.1 Virtualization and KVM

2.1.1 Hardware-Assisted Virtualization

Virtualization is a technology that enables multiple independent operating systems to run simultaneously on a single physical machine. By introducing a software layer called the hypervisor (or Virtual Machine Monitor, VMM) between the hardware and the guest OS, each guest OS is provided with a set of virtualized hardware resources (including CPU, memory, I/O devices, etc.) and operates in an isolated environment as if it has its own dedicated physical machine. This technology enables cloud service providers to consolidate workloads from multiple tenants onto a single physical host, achieving resource sharing and isolation.

To improve virtualization performance, processor vendors introduced hardware-assisted virtualization technologies (such as Intel VT-x and AMD-V), which natively integrate dedicated virtualization execution modes into the CPU, allowing guest operating systems to run at near-native performance [2]. When a guest executes certain sensitive operations, the processor automatically transfers control back to the hypervisor for handling; this event is called a VM-Exit. This hardware-level mechanism significantly improves virtualization performance and forms the foundation of modern virtualization architectures.

2.1.2 Kernel-based Virtual Machine (KVM)

Kernel-based Virtual Machine (KVM) is a virtualization module built into the Linux kernel that leverages the aforementioned hardware-assisted virtualization capabilities to turn the Linux kernel itself into a hypervisor [3][4]. KVM provides virtualization infrastructure as a kernel module, including vCPU creation and scheduling, guest physical memory management, and VM-Exit event handling. Userspace programs interact with KVM through ioctl interfaces (such as KVM_CREATE_VM, KVM_CREATE_VCPU, KVM_RUN, etc.) to create and control virtual machines [4].

Under the KVM architecture, the actual responsibility of constructing a complete virtualization environment falls on the userspace hypervisor program. These hypervisors are responsible for setting up the initial state of the VM, emulating peripheral devices (such as disks, network interfaces, display adapters, etc.), and handling operations that the guest cannot perform directly on the hardware. Major KVM hypervisors currently include QEMU [5], Firecracker [6], Cloud Hypervisor [7], crosvm, and kvmtool, each designed and optimized for different use cases.

2.1.3 VM-Exit Mechanism

The hypervisor drives guest execution by continuously repeating the following cycle: it calls the KVM_RUN ioctl to begin guest execution; when the guest performs certain specific operations, the processor pauses guest execution and transfers control back to KVM, an event known as a VM-Exit. After handling some VM-Exit events within the kernel, KVM passes events that require userspace intervention to the hypervisor program for processing; the hypervisor then calls KVM_RUN again, and the cycle repeats. In this architecture, each vCPU corresponds to a separate thread in the hypervisor process [4]. The VM-Exit types relevant to this study include:

- **KVM_EXIT_MMIO**: Triggered when the guest accesses an unmapped memory address, typically used for accessing MMIO device registers. Upon receiving this event, the hypervisor emulates the corresponding device behavior based on the accessed address and data.

- **KVM_EXIT_IO**: Triggered when the guest executes IN/OUT instructions to access I/O ports, used for Port I/O type device accesses. For example, PCI configuration space accesses in the x86 architecture are performed through the 0xCF8/0xCFC I/O ports.

These two VM-Exit mechanisms form the foundation for device emulation by the hypervisor: each access by the guest to a virtual device triggers a VM-Exit, which is intercepted and responded to by the hypervisor, thereby emulating complete hardware device behavior without the guest's knowledge. In this study, vmsh leverages these VM-Exit events to intercept and handle guest device access requests from outside the hypervisor.

2.2 VirtIO Standard and PCI Transport

2.2.1 VirtIO Overview

In traditional device emulation, the hypervisor reproduces the register interface of real hardware for the guest, and the guest uses the same drivers as in a physical environment to operate the devices. However, real hardware register interfaces are designed for physical environments; in a virtualized setting, each register access by the guest requires hypervisor intervention, and a single I/O operation often involves multiple register accesses, resulting in accumulated overhead that severely impacts performance [8].

Paravirtualized I/O takes a different approach: instead of operating devices through emulated hardware register interfaces, the guest uses drivers specifically designed for virtualized environments and exchanges data with the hypervisor directly through shared memory, significantly reducing the number of hypervisor interventions required [8]. However, different virtualization platforms implemented their own paravirtualized device interfaces, preventing drivers from being shared across platforms. VirtIO was proposed as a cross-platform common device interface standard to address this fragmentation [8].

VirtIO defines a unified interface specification for various virtual devices (such as block devices, network devices, etc.), enabling VirtIO drivers in the guest to interoperate with any standards-compliant hypervisor device implementation [9]. Currently, all major KVM hypervisors implement VirtIO devices, and major operating systems provide corresponding VirtIO drivers, making VirtIO the de facto standard for virtualized device emulation.

The core data exchange mechanism of VirtIO is the virtqueue. A virtqueue is a set of ring-shaped descriptor queues (descriptor rings) residing in shared memory between host and guest. The guest driver writes I/O requests into descriptors in the virtqueue and notifies the host-side device to process them; once the device completes processing, it writes results back to shared memory and notifies the guest via an interrupt. This shared-memory-based design avoids per-transaction VM-Exits, significantly improving I/O performance. Each VirtIO device may have one or more virtqueues depending on its type; for example, a block device typically uses a single

request queue, while a network device uses separate receive and transmit queues.

2.2.2 Transport Mechanism

In the VirtIO standard, the transport determines how the guest discovers devices and accesses device configuration interfaces. The VirtIO specification defines two primary transports: Virtio over PCI Bus and Virtio over MMIO [9].

MMIO transport maps device configuration registers to a memory address range, and the guest operates the device through memory access instructions. This approach does not rely on hardware infrastructure such as the PCI bus, is structurally simple, and is particularly prevalent in minimalist virtualization environments such as microVMs. vmsh's original device emulation uses MMIO transport [1].

PCI transport presents VirtIO devices as devices on the PCI bus, utilizing the PCI standard's configuration space, BAR mapping, and Capability mechanisms to carry the VirtIO configuration interface. Since x86 platforms universally support the PCI bus, most KVM hypervisors targeting general-purpose scenarios (such as QEMU) support PCI transport. This study introduces this transport mechanism to vmsh.

TAIPEI TECH

2.2.3 PCI Transport

PCI transport presents VirtIO devices as devices on the PCI bus. The following first introduces the relevant fundamentals of the PCI bus, then explains how VirtIO utilizes these mechanisms to implement its transport.

PCI Configuration Space and Access Mechanism

Each device on the PCI bus has a 256-byte Configuration Space. The operating system reads and writes the configuration space to identify the device's identity and type, configure system resources required by the device, and query the extended features supported by the device [10]. For example, the beginning of the configuration space stores the device's Vendor ID and Device ID, which the operating system uses to determine the device's manufacturer and model and load the corresponding driver.

In the x86 architecture, PCI configuration space access is performed through a pair of I/O ports: the guest first writes the target device's bus number (Bus), device number (Device), function number (Function), and register offset to I/O port 0xCF8 (CONFIG_ADDRESS), then reads or writes the corresponding configuration data through I/O port 0xCFC (CONFIG_DATA) [10]. In a KVM virtualization environment, guest accesses to these I/O ports trigger KVM_EXIT_IO, which is intercepted by the hypervisor to emulate the corresponding PCI device responses.

Base Address Register (BAR)

The configuration space contains up to six BARs (Base Address Registers), which declare the memory-mapped or I/O port resources required by the device. During device initialization, the operating system allocates address ranges for the device through BARs; the device's functional registers are then mapped within these ranges, and drivers subsequently access the device by operating on these ranges [10].

PCI Capability Structure

The PCI specification defines a Capability mechanism that allows devices to declare supported extended features within the configuration space. Capabilities are chained together as a linked list: the Capabilities Pointer (offset 0x34) in the configuration space points to the first Capability, and each Capability structure contains a next pointer pointing to the next one until the end of the list [10]. This mechanism allows PCI devices to flexibly extend their feature definitions within the framework of the standard configuration space.

VirtIO-PCI Capability Mapping

VirtIO PCI transport utilizes the above Capability mechanism to define a set of VirtIO-specific Capability structures within the PCI configuration space. Each Capability describes the location of a configuration region, specifying the BAR number and the offset within the BAR, serving as an index for the guest driver to locate each configuration region [9]. The VirtIO specification defines four primary configuration regions:

- **Common Configuration**: The device's general configuration interface, including feature negotiation, device status machine, and virtqueue configuration. The guest driver completes the device initialization process through this region.

- **Notification**: The guest driver writes to this region to notify the device to process new I/O requests.
- **ISR Status**: The device's interrupt status register; the guest driver reads this region to confirm the interrupt source.
- **Device-specific Configuration**: Configuration data specific to the device type, such as the storage capacity of a block device.

The actual data for these configuration regions resides in the memory range mapped by the BAR. After the guest driver obtains each region's address through the Capabilities, it can operate the device directly using memory access instructions.

In summary, VirtIO PCI transport builds device discovery and configuration on the foundation of the PCI standard: the configuration space and Capabilities allow the guest to discover devices and obtain the locations of configuration regions, while BAR mapping provides the interface for drivers to operate the device. In a virtualization environment, guest accesses to the configuration space trigger KVM_EXIT_IO type VM-Exits via Port I/O, while accesses to BAR ranges trigger KVM_EXIT_MMIO type VM-Exits; the hypervisor implements complete PCI device emulation by handling these two types of events.

2.3 VMSH: Hypervisor-agnostic Guest Overlays for VMs

vmsh is the system that this study extends. The following introduces its design goals and technical architecture.

2.3.1 Overview and Design Goals

In cloud environments, lightweight virtual machines (lightweight VMs) are widely adopted due to their compact image size and fast startup time. To achieve these advantages, lightweight VM images typically contain only the minimal kernel and userland required to run the target

application, omitting auxiliary tools such as monitoring and debugging utilities. However, when users need to add extra tools or services to a VM during runtime, the traditional approach requires rebuilding and redeploying the VM image, which not only undermines the advantages of the lightweight design but also requires restarting the VM, causing the loss of existing runtime state.

vmsh [1] is a hypervisor-agnostic abstraction layer that enables on-demand service attachment to running VMs. vmsh dynamically attaches additional applications to a VM through a file system image and uses container-based isolation to ensure that newly added services do not affect the existing applications within the VM. Users can interact with the newly attached services through the console interface provided by vmsh, without modifying the VM image or restarting the VM.

vmsh pursues three design goals [1]:

- **Non-cooperativeness**: vmsh does not rely on pre-installed agents or management programs in the guest userspace. Even when no cooperative software exists in the guest, vmsh can complete service attachment from the outside.
- **Generality**: vmsh does not depend on the API of any specific hypervisor but instead operates through the underlying KVM interface, enabling support for multiple KVM-based hypervisors. Additionally, vmsh supports a wide range of Linux kernel versions, without being constrained by specific kernel APIs.
- **Performance**: vmsh's attachment process should not degrade the performance of existing applications in the guest. The performance of attached tools and services is a secondary concern but must remain usable.

vmsh currently supports four KVM-based hypervisors: QEMU, Firecracker, crosvm, and kvmtool, and is compatible with multiple Linux LTS kernel versions [1].

2.3.2 System Architecture

As an independent host process, vmsh runs alongside the hypervisor on the same host machine. vmsh does not modify the hypervisor's source code, nor does it rely on any man-

agement API provided by the hypervisor. Instead, vmsh completes all operations externally through ptrace and the underlying KVM API. Figure 2.1 presents the system architecture of vmsh (adapted from [1] Figure 2), whose workflow can be divided into the following four steps, corresponding to steps 1–4 as labeled in the figure:

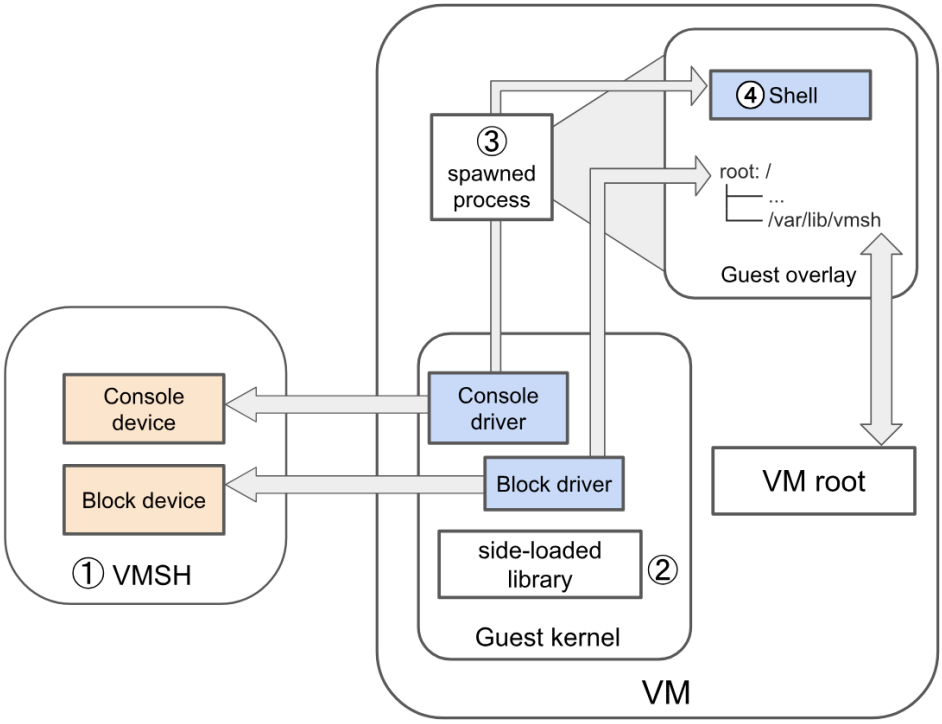

Figure 2.1 vmsh System Architecture (Adapted from [1])

In step (1) Attach, vmsh creates VirtIO block and console devices on the host side and attaches them to the target hypervisor process. The block device provides the file system image, and the console device handles input and output between the user and the VM.

In step (2) Side-load, vmsh loads a kernel library (called Stage1) into the guest kernel, enabling the guest to drive the virtual devices provided by vmsh. This step is accomplished by directly operating the KVM API, without requiring any pre-installed cooperative software in the guest.

In step (3) Setup, Stage1 activates the corresponding VirtIO drivers within the guest kernel, mounts the file system provided by the block device, and establishes a container-based isolation environment (guest overlay). The userspace program in this phase is called Stage2.

In step (4) Spawn, vmsh launches the user-specified program (e.g., a shell) within the con-

tainer environment. The program's input and output are routed back to the host through the console device, allowing the user to interact with the newly attached services in the VM.

2.3.3 Design and Key Techniques

Each phase of the above four-step workflow faces different technical challenges. The following introduces four core designs of vmsh, describing the challenges faced and the solutions adopted in each.

Hypervisor-agnostic Side-loading

In step (2), vmsh needs to load a kernel library into the guest kernel. However, the side-loading interfaces provided by different KVM-based hypervisors are inconsistent, and some hypervisors (such as Firecracker and kvmtool) do not provide such interfaces at all, making it impossible to load code in a unified manner.

vmsh's approach is to bypass the hypervisor's management interface entirely, tracing the hypervisor process via ptrace and directly operating the underlying KVM API. Specifically, vmsh locates the KVM file descriptor held by the hypervisor process through the Linux /proc filesystem, uses the KVM API to read the guest page table to understand the guest's memory layout, allocates available space in the high-address region of guest physical memory, loads the Stage1 kernel library into that region, and finally modifies the vCPU's instruction pointer to cause the guest to begin executing the loaded code [1].

Kernel-agnostic Library

The kernel library loaded in step (2) must be able to operate on different versions of the guest kernel. However, the internal APIs and data structures of the Linux kernel are not stable interfaces and may differ between versions, making it challenging to build a kernel library that works across versions.

vmsh addresses this challenge with two strategies. First, Stage1 is designed as a shared ELF, which dynamically locates required kernel function addresses at runtime through binary analysis. Specifically, vmsh traverses the guest page table to locate the actual loading position of the kernel, handling the randomization caused by KASLR, then parses the ksymtab data structure within the kernel to obtain the addresses of all exported functions, which are used to

fix up function references in Stage1. Second, Stage1 offloads as much functionality as possible to existing drivers within the guest kernel (such as VirtIO drivers), minimizing the use of kernel interfaces and reducing direct dependencies on internal kernel APIs.

Hypervisor-independent VirtIO Devices

The VirtIO devices created in step (1) run within the vmsh process rather than inside the hypervisor. Typical hypervisors (such as QEMU and Firecracker) implement device emulation within their own process and can directly access the device management interfaces provided by KVM. However, vmsh's devices reside outside the hypervisor process and cannot use the hypervisor's built-in device emulation framework, necessitating solutions to two problems: (1) how to intercept VMEXITs triggered by guest accesses to MMIO addresses, and (2) how to exchange data between host and guest through virtqueues located in guest virtual memory [1].

vmsh implements two types of VirtIO devices: a block device for providing the file system image, and a console device for redirecting application input and output. These devices are registered in the guest by the side-loaded kernel library (Stage1).

For MMIO interception, when the guest driver accesses the MMIO registers of a VirtIO device, the access triggers a VMEXIT because the corresponding address is not mapped to physical memory. vmsh intercepts the hypervisor's KVM_RUN ioctl via ptrace, intervenes when a VMEXIT occurs, and reads the memory-mapped vCPU file descriptor to parse and process the MMIO request. Additionally, vmsh supports ioregionfd as an alternative interception method, using file descriptor notifications instead of ptrace interception to reduce context switch overhead.

For interrupt notification, vmsh uses KVM's irqfd mechanism to send interrupts after device processing is complete, notifying the guest driver to retrieve the results.

vmsh's VirtIO devices are implemented using libraries from the rust-vmm project, with the backend extended to support cross-process memory access. The complete I/O flow is as follows: (1) the guest driver writes I/O requests to the driver virtqueue and writes to an MMIO register to notify vmsh, triggering a VMEXIT; (2) vmsh intercepts this VMEXIT, retrieves the request from the driver virtqueue, processes it, and writes the result to the vmsh virtqueue; (3) vmsh sends an interrupt via irqfd, notifying the guest driver to retrieve the result.

TAIPEI TECH

Container-based System Overlay

In step (3), vmsh needs to mount the file system provided by the block device into the guest. However, the mounted file system may conflict with the guest's existing root filesystem in terms of paths, for example, both may contain configuration files under /etc.

vmsh uses mount namespace isolation through Stage2 in the guest userspace. The file system provided by the block device is mounted as an overlay, serving as the root filesystem of the container environment, while the guest's original mount points are moved under the /var/lib/vmsh directory, ensuring mutual isolation.

This study extends the above vmsh architecture by implementing VirtIO-PCI transport for vmsh and replacing the irqfd interrupt mechanism with KVM_SIGNAL_MSI, enabling vmsh to support Cloud Hypervisor, which adopts the Split IRQChip architecture.

2.4 Interrupt Mechanisms in Virtualization

2.4.1 KVM Interrupt Delivery Mechanism

In a virtualization environment, KVM is responsible for delivering interrupts to the guest's vCPUs. The core component of interrupt delivery is the LAPIC (Local Advanced Programmable Interrupt Controller); KVM emulates one LAPIC for each vCPU, which receives interrupt signals and notifies the corresponding vCPU for handling [4].

Traditional pin-based interrupts are delivered through the IOAPIC (I/O Advanced Programmable Interrupt Controller). The IOAPIC receives interrupts from external devices and, based on its internal Redirection Table, determines the target vCPU and interrupt vector, then forwards the interrupt to the corresponding LAPIC.

KVM manages interrupt routing through the GSI routing table. The GSI routing table maps each GSI (Global System Interrupt) number to a specific interrupt delivery method, such as forwarding to a specific IOAPIC pin or directly delivering as an MSI message to the LAPIC.

KVM provides two IRQChip modes, which determine how interrupt controllers are managed [4]:

- **Full IRQChip**: Both IOAPIC and LAPIC are managed within the KVM kernel. The entire interrupt path from GSI routing through IOAPIC to LAPIC is completed within the kernel.
- **Split IRQChip**: The KVM kernel manages only the LAPIC and does not create an IOAPIC within the kernel. The IOAPIC may be implemented by the userspace hypervisor.

KVM also provides the irqfd mechanism, which allows external programs to trigger interrupt injection through an eventfd. When the eventfd is written to, KVM looks up the GSI number bound to that irqfd and consults the GSI routing table to deliver the interrupt to the corresponding target.

2.4.2 MSI/MSI-X Interrupts

Traditional pin-based interrupts are limited by the number of interrupt lines available on the IOAPIC, and all interrupts must be forwarded through the IOAPIC, which can become a bottleneck in multi-device environments. The PCI specification introduced MSI (Message Signaled Interrupts) and its extended version MSI-X to replace physical interrupt lines with a message-based approach [10].

MSI/MSI-X interrupts bypass the IOAPIC; instead, the device directly writes an MSI message to the target LAPIC. Each MSI message consists of two fields: the address specifies the target LAPIC's address, and the data contains the interrupt vector and delivery mode information.

MSI-X offers greater flexibility compared to MSI: each device can have up to 2048 independent interrupt vectors, each with its own address and data settings. These settings are stored in the MSI-X Table, which resides in the memory range mapped by the device's BAR and is configured by the driver during initialization.

In a KVM virtualization environment, MSI interrupts can be delivered through two methods [4]:

- **GSI routing**: The hypervisor creates a KVM_IRQ_ROUTING_MSI type entry in the GSI routing table, mapping a specific GSI number to the MSI message's address and data. When used with irqfd, triggering the eventfd causes KVM to write the MSI message to the LAPIC based on that entry.

- **KVM_SIGNAL_MSI**: Directly writes the MSI message to the LAPIC via ioctl, bypassing the GSI routing table. This method does not depend on the IRQChip mode configuration and works under both Full IRQChip and Split IRQChip.

TAIPEI TECH

Chapter 3 Proposed Method

3.1 System Architecture Overview

The device emulation of the original vmsh [1] is built upon the VirtIO MMIO transport. As shown in Figure 3.1, vmsh operates as an independent host process external to the hypervisor, using ptrace to trace the hypervisor process for device emulation and control. When the VirtIO driver within the guest accesses device registers in the MMIO region, a KVM_EXIT_MMIO VM-Exit is triggered, which vmsh intercepts via ptrace and handles the device-side response. For interrupt injection, vmsh uses KVM's irqfd mechanism: when the device needs to notify the guest, vmsh writes to an eventfd, and upon receiving the event, KVM routes the interrupt through the GSI routing table to the in-kernel IOAPIC, which then delivers it to the target vCPU. The in-kernel IOAPIC and GSI routing table are established by the hypervisor during startup via KVM_CREATE_IRQCHIP and KVM_SET_GSI_ROUTING; vmsh itself does not participate in building this infrastructure but rather directly uses and depends on its existence within the interrupt path.

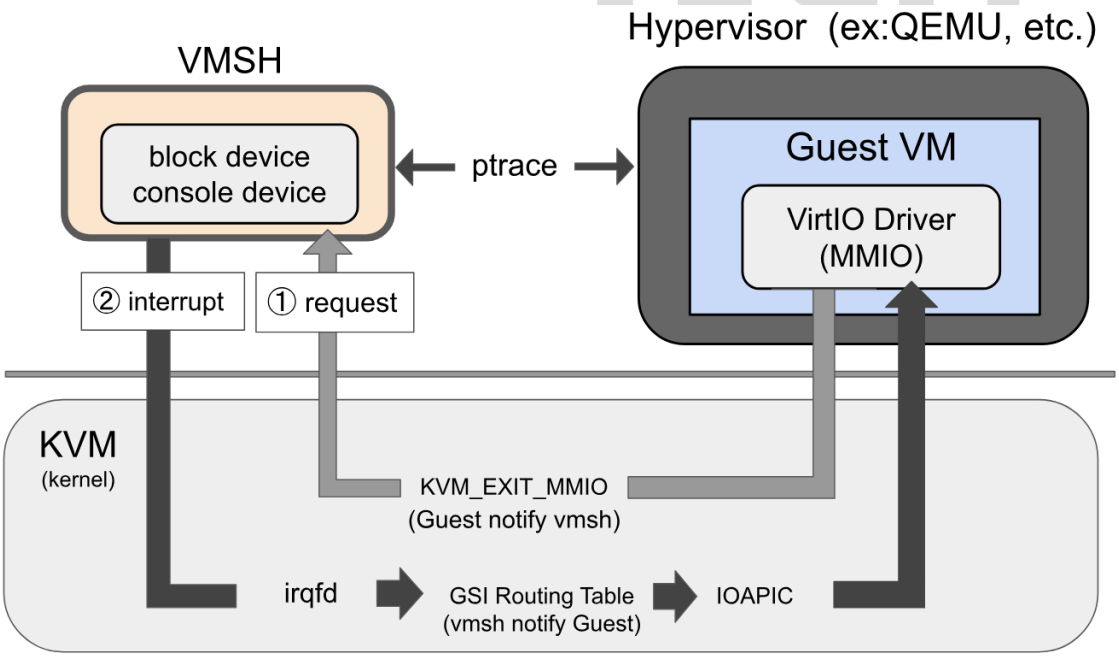

Figure 3.1 vmsh Original Architecture (VirtIO-MMIO Transport)

In Figure 3.1, (1) indicates the guest notifying vmsh to process I/O requests through the MMIO region, and (2) indicates vmsh delivering interrupts to the guest via irqfd through the GSI Routing Table and in-kernel IOAPIC. The path of (2) depends on the interrupt routing infrastructure established by the hypervisor, which does not exist on Cloud Hypervisor.

The above architecture works correctly on QEMU, Firecracker, crosvm, and kvmtool, as these hypervisors all adopt the Full IRQChip mode, where the KVM kernel internally manages the complete interrupt routing infrastructure, which is compatible with vmsh's existing implementation. However, Cloud Hypervisor adopts the Split IRQChip architecture, where the IOAPIC is implemented by Cloud Hypervisor in userspace; the KVM kernel contains no in-kernel IOAPIC and no corresponding GSI routing table. This means that the irqfd interrupt path on which vmsh's existing implementation depends does not exist on Cloud Hypervisor; after writing to the eventfd, the interrupt signal cannot be delivered to the guest due to the absence of the corresponding routing infrastructure. This renders vmsh's existing MMIO transport approach inapplicable on Cloud Hypervisor.

TAIPEI

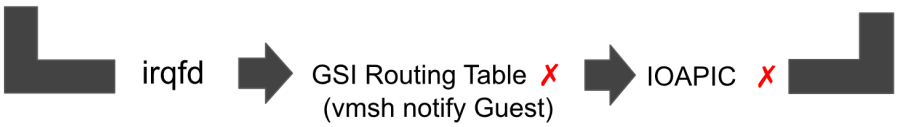

Figure 3.2 vmsh's Existing irqfd Interrupt Path Unavailable on Cloud Hypervisor (Split IRQChip)

To resolve the above incompatibility, an interrupt mechanism that does not depend on GSI routing or the IOAPIC is needed. This study uses the KVM_SIGNAL_MSI [4] ioctl to write interrupt messages directly to the vCPU's Local APIC, bypassing the GSI routing table and IOAPIC, thus working regardless of whether the hypervisor adopts a Full or Split IRQChip architecture.

KVM_SIGNAL_MSI is a KVM ioctl for sending MSI-X type interrupts, requiring the address and data of the interrupt vector in MSI-X format. MSI-X is an interrupt mechanism specific to PCI devices, with interrupt vectors stored in the PCI device's MSI-X Table; only PCI

devices possess this structure. Therefore, adopting KVM_SIGNAL_MSI necessarily requires the device to support MSI-X interrupts, which in turn means implementing, beyond the existing MMIO transport, a complete VirtIO-PCI transport for vmsh, enabling the guest to discover and access vmsh's virtual device through standard PCI mechanisms and receive MSI-X type interrupt notifications.

Based on the above analysis, this study proposes **Non-cooperative PCI Device Emulation**, introducing PCI transport to vmsh to resolve the interrupt path unavailability issue. Figure 3.3 shows the PCI transport architecture implemented in this study, contrasting with the MMIO transport in Figure 3.1. In Figure 3.3, (1) indicates the guest accessing PCI configuration space (0xCF8/0xCFC) via KVM_EXIT_IO and accessing BAR regions via KVM_EXIT_MMIO, both intercepted and handled by vmsh. (2) indicates vmsh delivering interrupts to the guest by writing directly to the LAPIC via KVM_SIGNAL_MSI, without depending on the GSI Routing Table and IOAPIC. Stage1 is responsible for triggering PCI rescan during VM runtime, enabling the guest to discover and initialize the virtual device. Items marked with a star (⋆) in the figure indicate components added by this study.

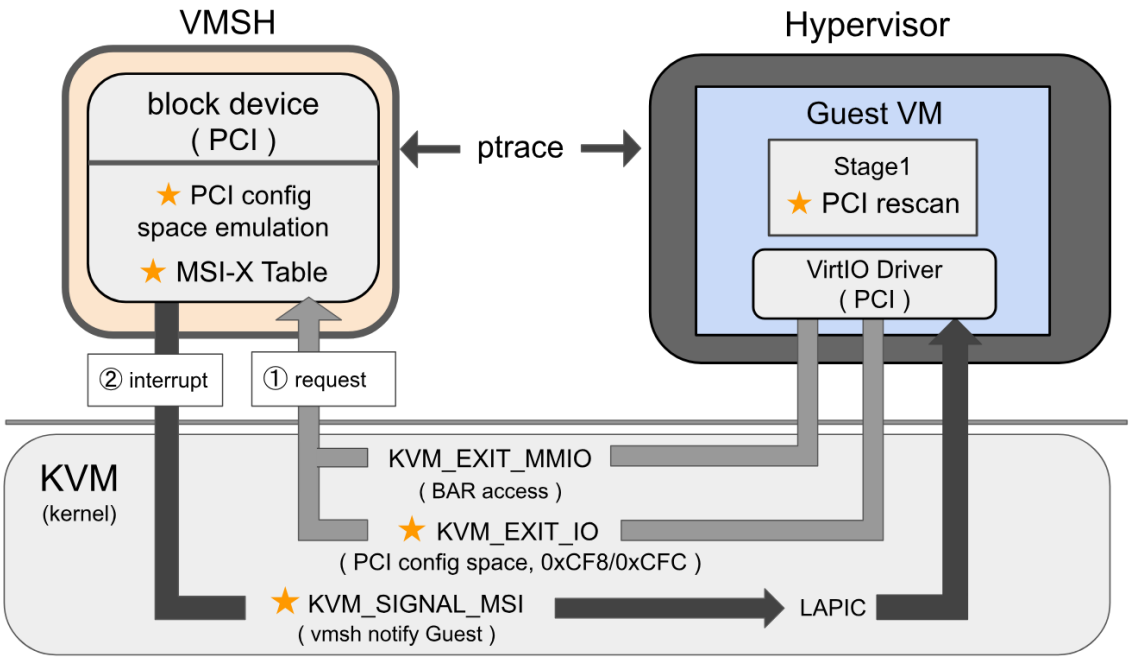

Figure 3.3 System Architecture (PCI Transport). Stars (⋆) Indicate Components Added by This Study.

As an extension of vmsh, the design of this study follows vmsh's two core principles:

- **Non-cooperativeness**: No reliance on programs pre-installed in the guest OS for cooperative operation.
- **Generality**: No dependence on any specific hypervisor's API; the design is not tied to any particular hypervisor's mechanisms and supports a wide range of guest kernel versions.

To implement PCI transport under these constraints, this study's design references and extends vmsh's existing implementation mechanisms, including its ptrace-based I/O interception framework and Stage1 injection mechanism, so that the PCI transport implementation extends vmsh's compatibility while maintaining vmsh's original non-cooperative and hypervisor-agnostic characteristics. The design goal of this study is to enable vmsh to perform device emulation through PCI transport and to be applicable to multiple KVM-based hypervisors.

As an independent process running alongside the hypervisor, vmsh's device emulation must be completed externally without hypervisor cooperation. Therefore, to implement non-cooperative PCI device emulation, the following core problems need to be addressed:

1. Device discovery. vmsh attaches devices dynamically during VM runtime, thus requiring a mechanism for the guest to discover newly attached virtual devices. This study extends vmsh's side-load library mechanism, leveraging its ability to load and execute code within the guest kernel to trigger PCI rescan, giving vmsh the capability to trigger PCI device initialization in the guest from outside the hypervisor (Section 3.2).

2. Port I/O interception. During PCI device initialization, the guest accesses the PCI configuration space through Port I/O instructions to interact with the virtual device. Since vmsh's existing implementation only has the capability to intercept MMIO type I/O accesses, and the hypervisor also processes these Port I/O accesses, vmsh's interception must avoid mutual interference with the hypervisor. This study applies concepts similar to vmsh's existing MMIO interception techniques to Port I/O interception, implementing and validating vmsh's ability to intercept and handle Port I/O accesses (Section 3.3).

3. PCI device emulation. After the guest discovers the device, it needs to complete initialization and perform I/O data exchange with the device through the standard VirtIO-PCI protocol.

vmsh's existing implementation only supports VirtIO-MMIO devices; this study implements VirtIO-PCI device emulation, including PCI configuration space, Capabilities structure, BAR mapping, and virtqueue setup, enabling the guest to communicate with the device emulated by vmsh through the standard VirtIO-PCI driver (Section 3.4).

4. Interrupt notification. After processing an I/O request, the device needs to notify the guest. As described above, vmsh's existing irqfd interrupt path is unavailable on Cloud Hypervisor; this study maintains the interrupt vectors written by the guest driver to the virtual PCI device's MSI-X Table and uses KVM_SIGNAL_MSI to deliver interrupts directly to the LAPIC, implementing a notification mechanism distinct from vmsh's existing interrupt path (Section 3.5).

5. I/O data path. To verify the feasibility and correctness of the above mechanisms, this study adopts a synchronous I/O model as a prototype, integrating device discovery, Port I/O interception, PCI device emulation, and interrupt notification to implement a complete data path from the guest issuing an I/O request to receiving an MSI-X interrupt response.

In summary, this study resolves the interrupt path incompatibility issue of vmsh on Cloud Hypervisor by introducing PCI transport, and implements a complete non-cooperative PCI device emulation scheme covering device discovery, Port I/O interception, PCI device emulation, and interrupt notification, while maintaining vmsh's original design philosophy. The following sections elaborate on each mechanism designed in this study, in the order of the device lifecycle.

3.2 PCI Bus Rescan and Device Discovery

vmsh needs to attach devices dynamically during VM runtime; however, the guest kernel has already completed PCI bus scanning during the boot phase and does not proactively rescan the PCI bus at runtime. Therefore, vmsh requires a mechanism that allows the guest to discover vmsh's PCI virtual device during VM runtime. This study extends and applies vmsh's Stage1 mechanism, using Stage1 to trigger the guest kernel to rescan the PCI bus, enabling the guest to detect newly attached virtual devices at runtime. The following describes how PCI rescan is triggered, and how the guest kernel discovers and initializes the virtual device injected by vmsh

during the rescan process.

3.2.1 PCI Rescan Trigger Mechanism

During VM boot, the guest kernel performs a full PCI bus scan, enumerating all Bus/Device/Function (BDF) slots and registering discovered devices with the system. This scan occurs before vmsh attach, so vmsh cannot insert virtual devices during the boot scan and requires a method to make the guest kernel rescan the PCI bus during VM runtime.

The Linux kernel provides the `/sys/bus/pci/rescan` interface; writing `1` to this file triggers a PCI bus rescan. This study extends vmsh's side-load library mechanism to utilize this interface: vmsh loads and executes code within the guest kernel through Stage1 (see Section 2.3), and this study implements a `trigger_pci_rescan()` function in Stage1 that opens `/sys/bus/pci/rescan` and writes `1` within the guest kernel context, triggering the guest kernel to rescan the PCI bus. Since Stage1 runs in guest kernel mode and has permission to access sysfs, it can directly operate this interface.

The key aspect of this method is that rescan differs in behavior from the cold boot scan. The cold boot scan enumerates all BDF slots, whereas rescan skips devices already registered in the system and only scans BDF slots that are not yet occupied. Figure 3.4 shows the simplified PCI rescan logic in the Linux kernel.

```
// drivers/pci/probe.c (simplified)
for (devfn = 0; devfn < 256; devfn += 8) {
    dev = pci_get_slot(bus, devfn);
    if (dev) {
        continue;  // Device already registered, skip
    }
    pci_scan_slot(bus, devfn);  // Scan only unregistered slots
}
```

Figure 3.4 Simplified PCI Rescan Logic in the Linux Kernel (drivers/pci/probe.c)

This behavior means that the rescan triggered by vmsh does not interfere with PCI devices already established by the hypervisor in the VM. For example, in a QEMU environment, devices created by the hypervisor such as VirtIO Block, VirtIO RNG, and ISA Bridge are already

registered during boot; rescan skips these registered BDF slots and only probes empty ones. Therefore, rescan is safe and does not affect normal VM operation.

3.2.2 Rescan Process and Virtual Device Discovery

During rescan, the guest kernel reads the Vendor ID (configuration space offset 0x00) of each unregistered BDF slot through the PCI configuration space access mechanism (0xCF8/0xCFC I/O ports). If the Vendor ID is 0xFFFF or 0x0000, the guest kernel determines that no device exists at that slot and skips it; if the value is valid, the guest kernel considers a device present and proceeds to read the full configuration space for device initialization.

vmsh selects in advance a BDF slot not used by the hypervisor as the address of the virtual device. When the guest kernel scans that slot during rescan, vmsh responds with a valid Vendor ID and Device ID through I/O interception techniques (detailed in Section 3.3), causing the guest kernel to recognize a device at that slot. For other empty slots, vmsh does not intervene, and the guest kernel receives 0xFFFF from the hypervisor and skips them normally.

Figure 3.5 illustrates this process using a QEMU environment as an example, assuming vmsh has selected BDF 00:0a.0 as the virtual device's slot.

```
rescan probes empty slots :

  BDF 00:05.0 → Guest reads Vendor ID
                → Receives 0xFFFF (empty slot)
                → Guest treats as no device, skip

  BDF 00:0a.0 → Guest reads Vendor ID
                → vmsh responds with Vendor ID=0x1AF4, Device ID=0x1041
                → Guest discovers VirtIO Block device
                → Guest continues reading configuration space
                → Guest loads VirtIO-PCI driver and starts device initialization

  BDF 00:0b.0 → Guest reads Vendor ID
                → Receives 0xFFFF (empty slot)
                → Guest treats as no device, skip
```

Figure 3.5 PCI Rescan Behavior with a vmsh Virtual PCI Device at BDF 00:0a.0

Upon receiving a valid Vendor ID, the guest kernel determines that a device exists at the BDF slot and proceeds with the subsequent PCI device initialization process, including reading

the full configuration space and allocating system resources required by the device. The specific details of this initialization process are described in Section 3.4.

3.3 Port I/O Interception Mechanism

Section 3.2 described how vmsh triggers PCI rescan to allow the guest to discover virtual devices. During the rescan process, the guest interacts with devices through the PCI configuration space access mechanism (0xCF8/0xCFC I/O ports), and vmsh needs to intercept these I/O accesses and respond with the virtual device's configuration data. This section first describes the two types of I/O exits in KVM, then explains how this study implements Port I/O interception and handling logic for vmsh, and finally describes how to prevent the hypervisor from overwriting vmsh's responses.

3.3.1 Two Types of I/O Exits

When the guest performs I/O operations, KVM generates a VM-Exit and records the relevant information in the kvm_run shared memory structure for the hypervisor to read and process. Depending on the I/O access method, KVM distinguishes between two exit types:

- **KVM_EXIT_MMIO**: Triggered when the guest accesses an unmapped memory address. The kvm_run structure records the physical address (phys_addr), data length (len), data content (data), and read/write direction (is_write). Device register accesses for VirtIO MMIO transport belong to this type.
- **KVM_EXIT_IO**: Triggered when the guest executes Port I/O instructions (IN/OUT). The kvm_run structure records the I/O port address (port), access direction (direction, distinguishing IN and OUT), data length (size), and the offset of the data within the kvm_run structure (data_offset). PCI configuration space accesses (0xCF8/0xCFC) belong to this type.

vmsh's existing implementation already has the ability to intercept KVM_EXIT_MMIO, used for handling device register accesses of VirtIO MMIO transport. However, PCI configura-

tion space accesses are performed through Port I/O, which triggers KVM_EXIT_IO, and vmsh did not originally have the ability to intercept this type of event. To implement PCI transport, this study adds KVM_EXIT_IO interception and handling mechanisms to vmsh.

3.3.2 Port I/O Interception Handling

vmsh's I/O interception is built upon the ptrace mechanism. vmsh uses ptrace to trace the thread in the hypervisor process responsible for executing the vCPU; when the hypervisor calls `ioctl(KVM_RUN)` to execute the vCPU and returns due to a VM-Exit, vmsh intercepts this event at the ptrace syscall-exit stop point. At this point, vmsh can read the kvm_run structure from the hypervisor process's memory via ptrace to obtain the VM-Exit type and related information.

As described in Section 2.2.3, x86 PCI configuration space access is performed through two I/O ports: the guest first writes the target BDF address to 0xCF8 (CONFIG_ADDRESS) using an `out` instruction, then reads or writes the configuration data at that location through 0xCFC (CONFIG_DATA) using an `in` or `out` instruction. In a KVM environment, these two Port I/O operations each trigger a separate KVM_EXIT_IO event, with the direction field in the kvm_run structure using IN and OUT to distinguish the type of instruction the guest executed.

This study implements the following KVM_EXIT_IO handling logic for vmsh, divided into two phases based on port: when vmsh intercepts an event with port = 0xCF8 and direction = OUT, it reads the value written by the guest from the data_offset location, parses the Bus/Device/Function and Register fields, and records the target BDF address the guest is currently accessing. When vmsh intercepts an event with port = 0xCFC, it checks whether the previously recorded BDF address corresponds to the virtual device's slot. If not, vmsh does not intervene and lets the hypervisor respond normally. If it is the virtual device, vmsh distinguishes between read and write based on direction: for direction = IN, vmsh writes the corresponding register value of the virtual device to the data_offset location in the kvm_run structure as the read response to the guest; for direction = OUT, vmsh reads the value written by the guest from the data_offset location and updates the virtual device's internal state.

3.3.3 Response Protection Mechanism

The handling logic in Section 3.3.2 enables vmsh to respond with the virtual device's configuration data; however, vmsh and the hypervisor both process the same KVM_EXIT_IO event, and without protection, the hypervisor's subsequent processing would overwrite the response vmsh has already written. This section explains the cause of this problem and its solution.

Since 0xCF8/0xCFC are standard PCI configuration space I/O ports, the hypervisor itself also processes these Port I/O events. After vmsh releases control, the hypervisor continues to process the same KVM_EXIT_IO event. For IN-direction accesses, the hypervisor queries its own managed PCI devices, and since vmsh's virtual device does not exist in the hypervisor's device list, the hypervisor treats the slot as empty and responds with 0xFFFF, overwriting the data vmsh previously wrote.

vmsh's existing MMIO interception has faced the same problem and solves it by modifying the direction field in the kvm_run structure: when responding to a KVM_EXIT_MMIO read request, after vmsh writes data to the data field of kvm_run, it changes `is_write` from 0 to 1, causing the hypervisor to treat the event as a write operation and perform no substantive processing, thereby avoiding overwriting vmsh's response. This study applies the same concept to Port I/O interception: after writing the response data to data_offset, vmsh changes the direction field in the kvm_run structure from IN to OUT. When the hypervisor subsequently processes this event, it sees an OUT-direction access; for an OUT instruction writing to a nonexistent PCI slot, the hypervisor performs no operation (no-op), thus preserving the data vmsh has written. On the next VM-Entry, KVM loads the data at the data_offset location into the guest's registers, allowing the guest to receive the virtual device configuration data responded by vmsh.

Figure 3.6 uses the example of the guest reading the virtual device's Vendor ID to contrast the behavior without and with the response protection mechanism. (a) If vmsh only writes the response data without modifying direction, the hypervisor overwrites vmsh's data with an empty slot response. (b) After vmsh changes direction from IN to OUT, the hypervisor's write to an empty slot produces no operation, and vmsh's response data is preserved.

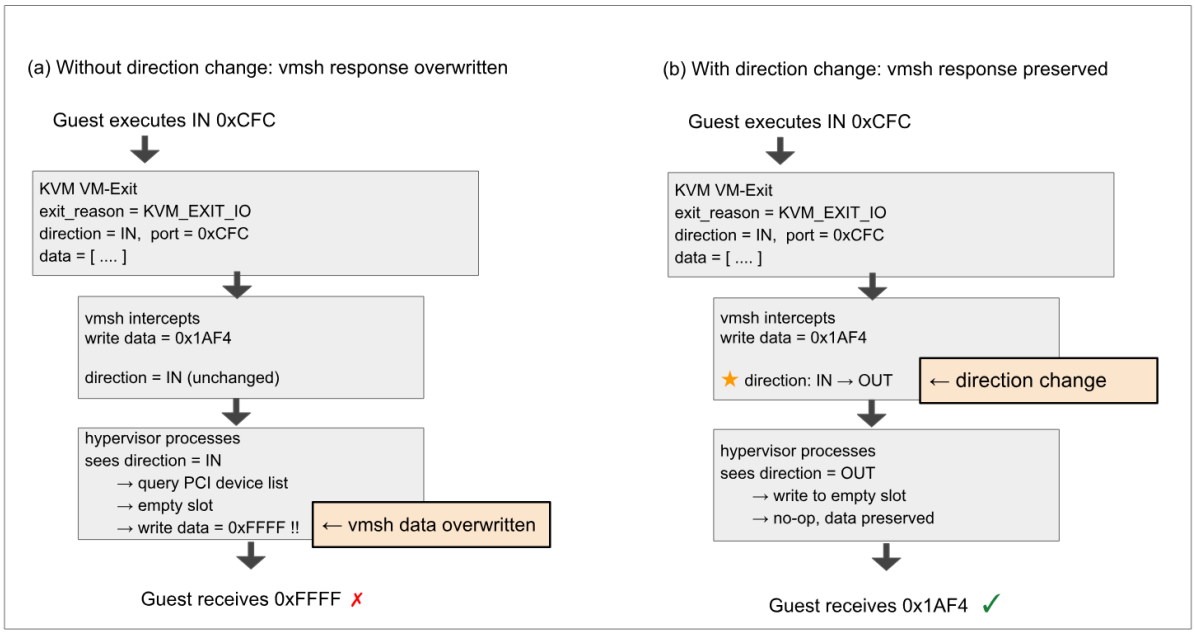

Figure 3.6 Illustration of Response Protection Mechanism

This response protection mechanism shares the same core idea as vmsh's existing MMIO interception: by modifying the direction field in the kvm_run structure, the hypervisor is prevented from performing substantive processing on the event, thereby avoiding overwriting the response data written by vmsh.

In summary, this study builds upon vmsh's existing ptrace infrastructure, specifically the ability to access the hypervisor's kvm_run shared memory structure and intercept VM-Exit events via ptrace, and implements a Port I/O interception and handling mechanism on top of this foundation. Through this mechanism, vmsh is able to allow the guest to access the PCI device configuration space emulated by vmsh in a non-cooperative manner.

3.4 VirtIO-PCI Device Emulation

Section 3.3 gave vmsh the ability to intercept the guest's PCI configuration space accesses. This section describes how vmsh emulates the VirtIO-PCI device's configuration space and connects to vmsh's existing MMIO interface, enabling the guest driver to communicate with vmsh and complete the PCI device initialization process.

3.4.1 Initialization Process Overview

From discovery by the guest to becoming ready for I/O, a VirtIO-PCI device goes through two initialization stages, each using a different access mechanism:

The first stage is completed through the PCI configuration space, using the Port I/O interception mechanism described above. The guest driver reads the device's basic information (Vendor ID, Device ID, Class Code, etc.) through the configuration space, then traverses the Capabilities linked list to identify the functions supported by the device and the BAR locations corresponding to each functional region. The PCI subsystem then probes the size of each BAR through the BAR sizing mechanism and allocates MMIO addresses for them. All accesses in this stage are performed through 0xCF8/0xCFC I/O ports, triggering KVM_EXIT_IO, which vmsh intercepts and responds with the virtual device's configuration data.

The second stage is performed through the BAR MMIO addresses allocated in the first stage. The guest driver accesses the registers of each functional region based on the BAR and offset information recorded in the Capabilities, completing the VirtIO handshake with vmsh: negotiating mutually supported features, setting up the virtqueue descriptor table and ring buffer addresses, and finally writing the DRIVER_OK flag to mark the device as ready. Accesses in this stage trigger KVM_EXIT_MMIO, which is handled by vmsh's existing MMIO interception mechanism.

After both stages are complete, the device enters the I/O processing phase.

3.4.2 PCI Configuration Space Stage

During the first stage of initialization, the guest driver interacts with the virtual device through the PCI configuration space, and vmsh needs to respond with correct configuration data for each read. This study implements a complete VirtIO-PCI configuration space for vmsh, covering three parts: device identification, Capabilities linked list, and BAR sizing.

Device identification. The guest driver first reads the device identification fields in the Configuration Space Header. The key fields responded by the virtual device include: Vendor ID (0x1AF4, Red Hat/VirtIO), Device ID (0x1041, VirtIO Block Device), Class Code (0x01, Mass

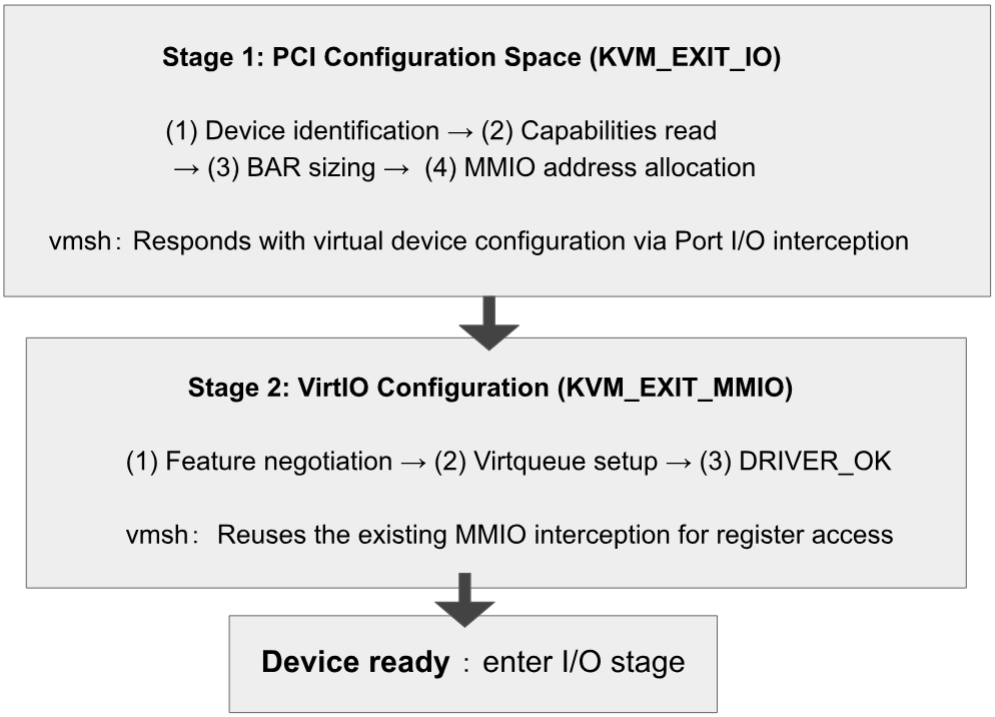

Figure 3.7 Two Stages of VirtIO-PCI Device Initialization

Storage Controller), and Header Type (0x00, standard device). These values cause the guest kernel to identify the device as a VirtIO block device and load the corresponding VirtIO-PCI driver.

Capabilities linked list. VirtIO-PCI devices describe the locations of their functional regions to the driver through the PCI Capabilities mechanism, where each Capability records the function type, the corresponding BAR (Base Address Register) number, and offset, directing the driver to access each functional region through the memory space mapped by the BAR. This study implements a linked list of five Capabilities for the virtual device, four of which are configuration regions defined by the VirtIO specification: COMMON_CFG (common configuration), NOTIFY_CFG (notification mechanism), ISR_CFG (interrupt status), and DEVICE_CFG (device-specific configuration), and one MSI-X Capability for declaring the device's MSI-X interrupt support. If the Capabilities content is incorrect or the linked list structure is corrupted, the VirtIO driver will abort initialization and the device will be unusable.

BAR sizing mechanism. The Capabilities linked list tells the driver which BAR each functional region corresponds to, but the guest PCI subsystem also needs to probe the memory size

required by each BAR through the BAR sizing mechanism before it can allocate MMIO addresses. This study implements the response logic for this mechanism in the virtual device, including predefining the size mask values for each BAR and recording the MMIO addresses ultimately allocated by the guest, enabling vmsh to determine which functional region the guest is accessing in the second stage and handle it accordingly.

Figure 3.8 shows the configuration space layout of the virtual PCI device implemented in this study. The upper portion contains PCI standard registers, and the lower portion contains the Capabilities linked list region, where five Capabilities are chained in sequence.

Offset	Register fields			
0x00	Vendor ID		Device ID	
0x04	Command		Status	
0x08	Revision ID / Class Code			
0x0C	Cache Line Size	Latency Timer	Header Type	BIST
0x10 – 0x24	BAR0 … BAR5			
0x34	Capabilities Pointer → 0x40			
0x3C	Interrupt Line / Pin			
Capability chain				
0x40	MSI-X Capability → next: 0x4C			
0x4C	COMMON_CFG (cfg_type=1) → next: 0x5C			
0x5C	NOTIFY_CFG (cfg_type=2) → next: 0x70			
0x70	ISR_CFG (cfg_type=3) → next: 0x80			
0x80	DEVICE_CFG (cfg_type=4) → next: 0x00 (end of chain)			

Figure 3.8 Virtual PCI Device Configuration Space Layout

3.4.3 VirtIO Configuration Stage

After the first stage is complete, the guest PCI subsystem has allocated MMIO addresses for each BAR. In the second stage, the guest driver accesses the registers of each functional region through MMIO based on the BAR and offset information recorded in the Capabilities, performing the VirtIO handshake with the virtual device. These accesses trigger KVM_EXIT_MMIO, which is handled by vmsh's existing MMIO interception mechanism.

The VirtIO handshake follows the state machine defined by the VirtIO specification, with the guest driver triggering state transitions at each stage by writing to the device_status register in the COMMON_CFG region. This study implements the following handshake process for the virtual device:

1. **Feature negotiation**: The guest driver reads the device feature bits advertised by the virtual device, selects the features mutually supported by both sides, writes the result to the driver feature bits, and sets the FEATURES_OK status bit. The virtual device verifies whether the features selected by the driver are valid; if not, it rejects FEATURES_OK and the driver aborts initialization. After successful negotiation, both sides establish the set of mutually supported features, and the driver proceeds to the virtqueue setup stage.

2. **Virtqueue setup**: The virtqueue is a shared memory structure for exchanging I/O requests and responses between the guest driver and the virtual device. The guest driver configures the virtqueue's descriptor table, available ring, and used ring in guest memory, and notifies the virtual device of these memory addresses through COMMON_CFG registers. The virtual device records these addresses for accessing the virtqueue in guest memory during subsequent I/O processing.

3. **DRIVER_OK**: After completing the above configuration, the guest driver writes the DRIVER_OK status bit, marking the device as ready. The virtual PCI device implemented in this study activates the device upon receiving DRIVER_OK, and the device then enters the I/O processing phase.

After the above two initialization stages, the guest has discovered and initialized the virtual

device emulated by vmsh through the standard VirtIO-PCI protocol, and subsequent I/O data exchange can proceed on this foundation.

3.5 MSI-X Interrupt Injection

After the device completes VirtIO-PCI initialization, it enters the I/O processing phase. When the device finishes processing an I/O request issued by the guest, it needs to notify the guest driver via an interrupt to retrieve the result. As described in Section 3.1, vmsh's existing irqfd interrupt path is unavailable under the Split IRQChip architecture; this study proposes using KVM_SIGNAL_MSI to deliver interrupts directly to the LAPIC, without depending on the GSI routing table and IOAPIC, enabling the interrupt mechanism to work under both Full and Split IRQChip architectures. This section describes the specific implementation of this interrupt path.

3.5.1 KVM_SIGNAL_MSI Interrupt Mechanism

KVM provides the KVM_SIGNAL_MSI interface, which allows the caller to deliver an interrupt directly to the target vCPU's LAPIC using an MSI message (address and data) as parameters. The address specifies the target LAPIC's address, and the data contains information such as the interrupt vector.

In vmsh's existing interrupt path, the irqfd mechanism depends on the GSI routing table and in-kernel IOAPIC established by the hypervisor during startup: the interrupt signal is routed through the GSI routing table to find the corresponding IOAPIC pin, which then forwards it to the target LAPIC. However, under the Split IRQChip architecture, this infrastructure does not exist within the KVM kernel, rendering vmsh's existing irqfd path inoperable. Although KVM provides the KVM_SET_GSI_ROUTING interface for updating the routing table, and vmsh could architecturally invoke it via ptrace injection, the semantics of this interface is to replace the entire routing table rather than add a single entry, and KVM does not provide an API to read the existing routing table. vmsh cannot know the hypervisor's existing entries for merging; writing blindly would overwrite the hypervisor's routing configuration, causing interrupts for

other devices to fail.

In contrast, KVM_SIGNAL_MSI writes interrupts directly to the LAPIC, each call is independent, does not require pre-established entries in the GSI routing table, and does not require the existence of an in-kernel IOAPIC. The MSI message itself contains complete interrupt delivery information, and each call only needs the parameters to complete interrupt injection. This study adopts KVM_SIGNAL_MSI as the interrupt injection method used by PCI transport, leveraging its independence from interrupt routing infrastructure to enable vmsh to deliver interrupts to the guest even in environments where the irqfd path is unavailable.

When using KVM_SIGNAL_MSI, the caller needs to provide the address and data of the MSI message as parameters. In a standard VirtIO-PCI device, the guest driver writes the address and data of the interrupt vector to the device's MSI-X Table during initialization; vmsh needs to intercept these writes and maintain the corresponding interrupt vector parameters for use during subsequent interrupt injection.

3.5.2 MSI-X Table and Interrupt Vector Management

The MSI-X Table is a structure used by PCI devices to store interrupt vectors, located within the memory region mapped by the device's BAR. Each table entry records the address and data of one interrupt vector; the guest driver writes interrupt vectors to this table via MMIO during device initialization, informing the device which target LAPIC to deliver interrupts to and which vector to use.

For vmsh, the MSI-X Table is the source for obtaining interrupt vector parameters. This study maps the MSI-X Table region to BAR4 of the PCI virtual device. When the guest driver writes address and data values via MMIO, a KVM_EXIT_MMIO is triggered; vmsh intercepts it and records the address and data into the MSI-X Table structure maintained internally by the virtual device. When the device subsequently needs to send an interrupt, vmsh retrieves the corresponding interrupt vector parameters from this structure and uses them as input parameters for KVM_SIGNAL_MSI to deliver the interrupt to the guest through KVM.

TAIPEI TECH

3.5.3 Interrupt Injection Implementation

KVM_SIGNAL_MSI is a KVM ioctl and therefore needs to be invoked through the VM file descriptor (vm_fd). However, vm_fd is obtained from KVM by the hypervisor when creating the VM; vmsh, as an external process, does not hold this file descriptor and therefore cannot directly invoke KVM_SIGNAL_MSI.

Therefore, this study utilizes vmsh's existing ptrace syscall injection mechanism to inject the ioctl call into a hypervisor vCPU thread that has been suspended by ptrace, borrowing the vm_fd held by the hypervisor to execute KVM_SIGNAL_MSI. The specific procedure is as follows: vmsh first assembles the address and data recorded in the MSI-X Table into a kvm_msi structure and writes it to the hypervisor's address space; it then saves the thread's registers and the original instruction at the instruction pointer location, writes a syscall instruction at the instruction pointer position, and sets the registers to the ioctl parameters (vm_fd, KVM_SIGNAL_MSI, address of the kvm_msi structure); vmsh then advances thread execution via ptrace, causing the kernel to execute the ioctl and complete the interrupt injection; finally, it restores the original instruction and registers, returning the hypervisor thread to its pre-injection state.

The ptrace syscall injection method described above is the preliminary implementation adopted by this study for interrupt triggering. As part of the PCI transport synchronous I/O model, this approach is implemented in a simpler synchronous manner to verify whether the interrupt path of PCI transport can operate correctly.

Figure 3.9 shows a complete I/O round-trip after the device is ready. (1) The guest driver writes to NOTIFY_CFG to initiate an I/O request, triggering KVM_EXIT_MMIO. (2) vmsh intercepts it via ptrace, reads the virtqueue, and executes disk I/O. (3) Upon completion, vmsh retrieves the interrupt vector parameters from the MSI-X Table and injects KVM_SIGNAL_MSI into the hypervisor vCPU thread via ptrace syscall injection; KVM writes the interrupt to the LAPIC. (4) vmsh then restores the thread state and resumes the vCPU. (5) Upon VM entry, the vCPU delivers the pending interrupt, and the guest retrieves the I/O result after receiving the interrupt.

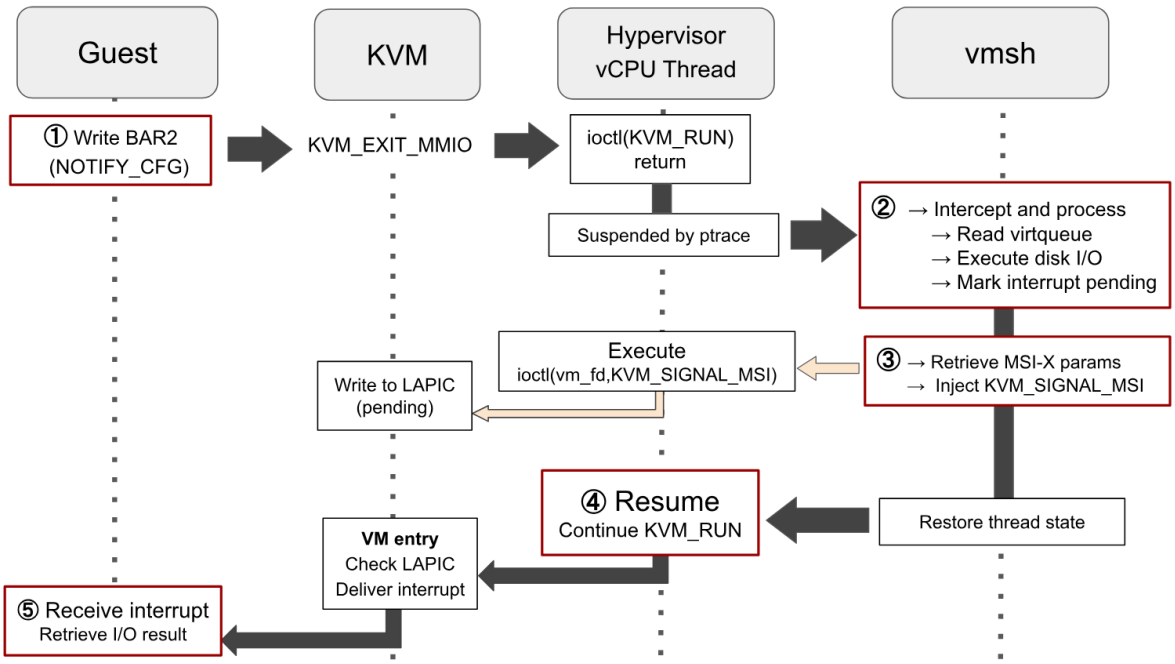

Figure 3.9 A Complete I/O Round-Trip

This study integrates the mechanisms from Section 3.2 through Section 3.5 using the synchronous I/O model described above, covering device discovery, Port I/O interception, PCI device emulation, and MSI-X interrupt injection, to implement a complete data path from the guest issuing an I/O request to receiving an interrupt response, verifying the feasibility of the non-cooperative PCI device emulation scheme.

Chapter 4 Experiment and Result

4.1 Functional Validation and Generality

4.1.1 PCI Device Discovery and Initialization

```
[root@qemu-microvm:~]# dmesg | tail -25
[   47.007513] stage1: detected linux version 6.3.6
[   47.007514] stage1: using PCI transport (EXPERIMENTAL)
[   47.007514] stage1: [EXPERIMENTAL] triggering PCI bus rescan
[   47.009020] pci 0000:00:0a.0: [1af4:1042] type 00 class 0x010000
[   47.009710] pci 0000:00:0a.0: reg 0x10: [mem 0x00000000-0x00000fff]
[   47.010314] pci 0000:00:0a.0: reg 0x18: [mem 0x00000000-0x00000fff]
[   47.010921] pci 0000:00:0a.0: reg 0x20: [mem 0x00000000-0x00000fff]
[   47.011512] pci 0000:00:0a.0: reg 0x30: [mem 0x00000000-0x000007ff pref]
[   47.021659] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021663] pci 0000:00:0a.0: BAR 0: assigned [mem 0x20000000-0x20000fff]
[   47.021777] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021779] pci 0000:00:0a.0: BAR 2: assigned [mem 0x20001000-0x20001fff]
[   47.021897] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.021898] pci 0000:00:0a.0: BAR 4: assigned [mem 0x20002000-0x20002fff]
[   47.022007] resource: avoiding allocation from e820 entry [mem 0x0009fc00-0x000fffff]
[   47.022008] pci 0000:00:0a.0: BAR 6: assigned [mem 0x20003000-0x200037ff pref]
[   47.022869] virtio-pci 0000:00:0a.0: enabling device (0000 -> 0002)
```

Figure 4.1 dmesg log of PCI device discovery and BAR assignment (QEMU)

```
[   47.052456] virtio_blk virtio2: [vdb] 131072 512-byte logical blocks (67.1 MB/64.0 MiB)
```

Figure 4.2 dmesg log of VirtIO negotiation completion (QEMU)

```
[root@qemu-microvm:~]# ls /dev/vd*
/dev/vda  /dev/vdb

[root@qemu-microvm:~]# cat /sys/block/vdb/size
131072
```

Figure 4.3 Guest userspace query of the vdb device

4.1.2 Block I/O and Filesystem Validation

4.1.3 Cloud Hypervisor Validation

```
[root@qemu-microvm:~]# dd if=/dev/vdb of=/dev/null bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00445993 s, 918 kB/s

[root@qemu-microvm:~]# dd if=/dev/zero of=/dev/vdb bs=4k count=1
1+0 records in
1+0 records out
4096 bytes (4.1 kB, 4.0 KiB) copied, 0.00578631 s, 708 kB/s

[root@qemu-microvm:~]# echo "VMSH_TEST_DATA" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
15 bytes copied, 0.0124918 s, 1.2 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_TEST_DATA
512 bytes copied, 0.00840111 s, 60.9 kB/s
```

Figure 4.4 Block I/O read/write and data integrity test (QEMU)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
fd0   2:0    1     4K  0 disk
sr0  11:0    1  1024M  0 rom
vda 254:0    0 267.5M  1 disk /nix/store
                               /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt
```

Figure 4.5 ext4 filesystem mount test (QEMU)

```
[root@qemu-microvm:~]# echo "VMSH_CH_TEST_2026" | dd of=/dev/vdb bs=512 count=1
0+1 records in
0+1 records out
18 bytes copied, 0.00511157 s, 3.5 kB/s

[root@qemu-microvm:~]# dd if=/dev/vdb bs=512 count=1 | head -c 20
1+0 records in
1+0 records out
VMSH_CH_TEST_2026
512 bytes copied, 0.00541082 s, 94.6 kB/s
```

Figure 4.6 Data integrity test (Cloud Hypervisor)

```
[root@qemu-microvm:~]# lsblk
NAME
    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
vda 254:0    0 267.5M  1 disk /nix/store
                              /
vdb 254:16   0    10M  0 disk

[root@qemu-microvm:~]# mkdir -p /tmp/mnt

[root@qemu-microvm:~]# mount /dev/vdb /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
bin  dev  etc  lost+found  nix  proc  sys  tmp  var

[root@qemu-microvm:~]# umount /tmp/mnt

[root@qemu-microvm:~]# ls /tmp/mnt
```

Figure 4.7 ext4 filesystem mount test (Cloud Hypervisor)

References

[1] Jörg Thalheim et al. “vmsh: Hypervisor-agnostic Guest Overlays for VMs”. In: *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys ’22)*. Rennes, France: ACM, 2022, pp. 678–693.

[2] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3: System Programming Guide*. 2026. URL: `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html`.

[3] Avi Kivity et al. “KVM: The Linux Virtual Machine Monitor”. In: *Proceedings of the Linux Symposium*. Ottawa, Canada, 2007, pp. 225–230.

[4] The Linux Kernel Community. *The Definitive KVM (Kernel-based Virtual Machine) API Documentation*. 2026. URL: `https://docs.kernel.org/virt/kvm/api.html`.

[5] Fabrice Bellard. “QEMU, a Fast and Portable Dynamic Translator”. In: *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. Anaheim, CA, USA, 2005, pp. 41–46.

[6] Alexandru Agache et al. “Firecracker: Lightweight Virtualization for Serverless Applications”. In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’20)*. Santa Clara, CA, USA, 2020, pp. 419–434.

[7] Cloud Hypervisor Contributors. *Cloud Hypervisor*. 2026. URL: `https://www.cloudhypervisor.org/`.

[8] Rusty Russell. “virtio: Towards a De-Facto Standard for Virtual I/O Devices”. In: *ACM SIGOPS Operating Systems Review* 42.5 (2008), pp. 95–103. DOI: `10.1145/1400097.1400108`.

[9] Michael S. Tsirkin and Cornelia Huck. *Virtual I/O Device (VIRTIO) Version 1.1*. OASIS. 2019. URL: `https://docs.oasis-open.org/virtio/virtio/v1.1/virtio-v1.1.html`.

[10] PCI-SIG. *PCI Local Bus Specification, Revision 3.0*. 2004.

TAIPEI TECH